

ATUALIZAÇÃO, REMOÇÃO, JOINS E CRUD COM PYTHON

Manipulação Completa de Dados e Relacionamentos

Duração:	120 minutos
Nível:	Intermediário
Tecnologias:	Python 3, SQLite, SQL
Pré-requisitos:	SELECT, WHERE, ORDER BY, funções de agregação

SUMÁRIO

1. Revisão Rápida: CREATE, INSERT, SELECT
2. Atualização de Dados com UPDATE
 - 2.1. Sintaxe e Exemplos Básicos
 - 2.2. UPDATE com WHERE — Atualizando Registros Específicos
 - 2.3. Boas Práticas e Armadilhas
3. Remoção de Dados com DELETE
 - 3.1. Sintaxe e Exemplos
 - 3.2. DELETE com WHERE — Removendo Registros Específicos
 - 3.3. Truncate vs DELETE — Quando Usar Cada Um
4. Integridade Referencial: Chaves Primárias e Estrangeiras
 - 4.1. Revisão de PK e FK
 - 4.2. ON DELETE e ON UPDATE — Comportamento em Cascata
5. Relacionamentos entre Tabelas: JOINS
 - 5.1. O Problema de Dados em Múltiplas Tabelas
 - 5.2. INNER JOIN — Apenas Registros com Correspondência
 - 5.3. LEFT JOIN — Todos da Esquerda
 - 5.4. RIGHT JOIN — Todos da Direita
 - 5.5. Comparativo Visual dos JOINS
 - 5.6. JOINS com Filtros (WHERE e ORDER BY)
6. CRUD Completo no Terminal com Python
 - 6.1. Estrutura de um CRUD
 - 6.2. Camada de Banco de Dados (db.py)
 - 6.3. Menu Interativo via input()
7. Atividade Prática: Sistema de Condomínio
 - 7.1. Modelagem: Garagens, Apartamentos e Proprietários
 - 7.2. Script Completo do Sistema
8. Referências e Próximos Passos

1. REVISÃO RÁPIDA: CREATE, INSERT, SELECT

Antes de avançar, vamos consolidar o que já sabemos. Nas aulas anteriores aprendemos a criar tabelas, inserir dados e consultá-los com filtros, ordenação e funções de agregação. Nesta aula vamos completar o ciclo de vida dos dados aprendendo a **atualizá-los** e **removê-los**, além de relacionar informações de múltiplas tabelas com **JOINS**.

Resumo dos comandos já estudados:

Comando	O que faz	Exemplo resumido
CREATE TABLE	Cria a estrutura da tabela	CREATE TABLE alunos (id INTEGER PRIMARY KEY ...)
INSERT INTO	Insere um novo registro	INSERT INTO alunos (nome) VALUES ('Ana')
SELECT	Consulta registros	SELECT nome, nota FROM alunos WHERE nota > 7
WHERE	Filtra registros	WHERE cidade = 'Salvador'
ORDER BY	Ordena resultados	ORDER BY nota DESC
COUNT/AVG/...	Funções de agregação	SELECT COUNT(*), AVG(nota) FROM alunos

Hoje aprenderemos: UPDATE (atualizar), DELETE (remover), conceitos de integridade referencial, e como juntar tabelas com JOINS — fechando assim o **ciclo CRUD** (Create, Read, Update, Delete).

2. ATUALIZAÇÃO DE DADOS COM UPDATE

O comando **UPDATE** modifica valores já existentes em uma ou mais colunas de registros que satisfaçam uma condição. Se você omitir o **WHERE**, **todos** os registros serão atualizados — o que raramente é o que você quer.

2.1. Sintaxe e Exemplos Básicos

```
-- Sintaxe geral
UPDATE nome_tabela
SET coluna1 = novo_valor1,
    coluna2 = novo_valor2
WHERE condição;

-- Atualizar UM campo de UM registro
UPDATE alunos
SET nota = 9.5
WHERE id_aluno = 3;

-- Atualizar VÁRIOS campos de UM registro
UPDATE alunos
SET nota      = 8.0,
    cidade    = 'Brasília',
    email     = 'novo@email.com'
WHERE id_aluno = 7;

-- Atualizar TODOS os registros (sem WHERE – use com cuidado!)
UPDATE produtos
SET preco = preco * 1.10;    -- reajuste de 10% em todos
```

2.2. UPDATE com WHERE — Atualizando Registros Específicos

```
import sqlite3

conexao = sqlite3.connect('escola.db')
cursor = conexao.cursor()

# ■■■ Exemplo 1: corrigir nota de um aluno específico ■■■■■■■■■■■■■■■■
cursor.execute('''
    UPDATE alunos
    SET nota = 8.5
    WHERE nome = 'Daniel Lima'
''')
print(f"Linhas afetadas: {cursor.rowcount}") # cursor.rowcount informa
                                              # quantos registros mudaram
conexao.commit()

# ■■■ Exemplo 2: atualizar cidade de vários alunos com IN ■■■■■■■■■■■■■■
cursor.execute('''
    UPDATE alunos
    SET cidade = 'Região Metropolitana de Salvador'
    WHERE cidade IN ('Lauro de Freitas', 'Camaçari', 'Simões Filho')
''')
print(f"Alunos transferidos: {cursor.rowcount}")
conexao.commit()

# ■■■ Exemplo 3: dar desconto em produtos com estoque baixo ■■■■■■■■■■■■■■
cursor.execute('''
    UPDATE produtos
    SET preco = preco * 0.85
    WHERE estoque < 5
''')
print(f"Produtos com desconto: {cursor.rowcount}")
conexao.commit()

# ■■■ Exemplo 4: atualizar usando valor calculado ■■■■■■■■■■■■■■■■
cursor.execute('''
    UPDATE alunos
    SET nota = nota + 0.5
    WHERE nota BETWEEN 6.0 AND 6.9
''')
print(f"Alunos que receberam 0.5 de bônus: {cursor.rowcount}")
conexao.commit()

conexao.close()
```

■■■ **Regra de Ouro:** Antes de executar um UPDATE, faça um SELECT com a mesma condição WHERE para confirmar quais registros serão afetados. UPDATE sem WHERE modifica TODOS os registros da tabela!

Comando	O que remove	Mantém estrutura?	Quando usar
DELETE	Registros (linhas)	Sim	Remover dados específicos ou todos preservando a tabela
DROP TABLE	A tabela inteira	Não	Eliminar definitivamente a tabela e seus dados
TRUNCATE *	Todos os registros	Sim	Limpar tabela rapidamente (não disponível no SQLite)

* No *SQLite* não existe *TRUNCATE*. Use *DELETE FROM tabela; (sem WHERE)* para o mesmo efeito.

■ **Cuidado com integridade referencial:** ao tentar deletar um registro que é referenciado por uma chave estrangeira (FK) em outra tabela, o banco pode retornar um erro. Abordaremos esse comportamento na próxima seção.

4. INTEGRIDADE REFERENCIAL: CHAVES PRIMÁRIAS E ESTRANGEIRAS

Bancos relacionais garantem que os dados permaneçam consistentes através das **restrições de integridade referencial**. Antes de estudarmos os JOINS, precisamos revisar — e aprofundar — como funcionam as chaves e o que acontece quando tentamos atualizar ou remover registros relacionados.

4.1. Revisão de Chave Primária (PK) e Chave Estrangeira (FK)

Conceito	Definição	Restrição principal
PRIMARY KEY (PK)	Identificador único de cada registro	Valor único e NOT NULL; geralmente AUTOINCREMENT
FOREIGN KEY (FK)	Coluna que referencia a PK de outra tabela	Tabela precisa existir na tabela pai (ou ser NULL)
UNIQUE	Garante que não haja duplicatas em uma coluna	Pode ser NULL (diferente de PK)
NOT NULL	Proíbe a ausência de valor	Campo sempre deve ter um valor

```
-- Exemplo de tabelas com relacionamento PK → FK
CREATE TABLE departamentos (
  id_dep  INTEGER PRIMARY KEY AUTOINCREMENT,
  nome_dep VARCHAR(80) NOT NULL,
  andar  INTEGER
);

CREATE TABLE funcionarios (
  id_func  INTEGER PRIMARY KEY AUTOINCREMENT,
  nome     VARCHAR(100) NOT NULL,
  salario  REAL,
  id_dep   INTEGER,                    -- coluna FK
  FOREIGN KEY (id_dep)
    REFERENCES departamentos(id_dep)  -- aponta para a PK da outra tabela
    ON DELETE SET NULL                -- se o depto for deletado, id_dep vira NULL
    ON UPDATE CASCADE                 -- se o id do depto mudar, propaga aqui
);
```

4.2. ON DELETE e ON UPDATE — Comportamento em Cascata

Quando excluímos ou atualizamos um registro pai (aquele cuja PK é referenciada), podemos instruir o banco sobre o que fazer nos registros filhos:

Opção	Comportamento quando o registro pai é alterado/removido
RESTRICT (padrão)	Lança erro — proíbe a operação enquanto houver filhos
CASCADE	Propaga a ação (delete ou update) automaticamente nos filhos
SET NULL	Define a FK dos filhos como NULL
SET DEFAULT	Define a FK dos filhos com o valor DEFAULT declarado
NO ACTION	Semelhante a RESTRICT, verificado ao final da transação

```
import sqlite3

# Ativar suporte a FK no SQLite (necessário em cada conexão!)
conexao = sqlite3.connect('empresa.db')
conexao.execute('PRAGMA foreign_keys = ON')  # ← obrigatório no SQLite
cursor = conexao.cursor()

# Tentar deletar um departamento com funcionários → erro com RESTRICT
```



```
try:
    cursor.execute('DELETE FROM departamentos WHERE id_dep = 1')
    conexao.commit()
except sqlite3.IntegrityError as e:
    print(f"Erro de integridade: {e}")
    # FOREIGN KEY constraint failed

conexao.close()
```

Importante: No SQLite, as restrições de FK **não** são ativas por padrão. Você precisa executar `PRAGMA foreign_keys = ON` em cada conexão.

5. RELACIONAMENTOS ENTRE TABELAS: JOINS

5.1. O Problema de Dados em Múltiplas Tabelas

Sistemas reais distribuem informações em várias tabelas para evitar redundância. Porém, ao consultar dados, frequentemente precisamos cruzar essas tabelas. Os **JOINS** fazem exatamente isso: combinam linhas de duas ou mais tabelas com base em uma coluna relacionada.

```
-- Problema: exibir Nome do aluno E nome da turma em que está matriculado
-- Sem JOIN você faria duas consultas separadas:
SELECT id_turma FROM alunos WHERE nome = 'Ana Silva';    -- retorna: 2
SELECT nome_turma FROM turmas WHERE id_turma = 2;        -- retorna: 'Turma B'

-- Com JOIN você faz em UMA consulta:
SELECT a.nome, t.nome_turma
FROM alunos a
INNER JOIN turmas t ON a.id_turma = t.id_turma
WHERE a.nome = 'Ana Silva';
```

5.2. INNER JOIN — Somente Registros com Correspondência

O **INNER JOIN** (ou simplesmente **JOIN**) retorna apenas as linhas que possuem correspondência em **ambas** as tabelas. Registros sem par são excluídos do resultado.

[illegible]

5.3. LEFT JOIN — Todos os Registros da Tabela da Esquerda

O **LEFT JOIN** retorna **todos** os registros da tabela à esquerda (antes do JOIN) e, quando disponível, os dados da tabela à direita. Quando não há correspondência, as colunas da direita aparecem como NULL.

```
Sintaxe
SELECT colunas
FROM tabela_esquerda
LEFT JOIN tabela_direita ON tabela_esquerda.fk = tabela_direita.pk;

-- ■■ Exemplo: TODAS as turmas, mesmo as sem alunos ██████████
SELECT t.nome_turma,
       a.nome      AS aluno,
       a.nota
FROM turmas t
LEFT JOIN alunos a ON t.id_turma = a.id_turma
ORDER BY t.nome_turma, a.nome;
-- Turmas sem alunos aparecem com NULL nas colunas de 'alunos'

-- ■■ Exemplo: encontrar turmas SEM alunos matriculados ██████████
SELECT t.nome_turma
FROM turmas t
LEFT JOIN alunos a ON t.id_turma = a.id_turma
WHERE a.id_aluno IS NULL;
-- WHERE filtra as linhas onde a direita é NULL → só turmas vazias

import sqlite3

conexao = sqlite3.connect('escola.db')
cursor  = conexao.cursor()

print("TODAS AS TURMAS (com ou sem alunos) – LEFT JOIN")
print("-" * 55)

cursor.execute('''
    SELECT t.nome_turma,
           COUNT(a.id_aluno) AS qtd_alunos,
           AVG(a.nota)      AS media_nota
    FROM turmas t
    LEFT JOIN alunos a ON t.id_turma = a.id_turma
    GROUP BY t.id_turma, t.nome_turma
    ORDER BY t.nome_turma
''')

print(f"{t.turma}<25} {t.alunos}<10} {t.media}")
print("-" * 45)
for row in cursor.fetchall():
    media = f"{row[2]:.2f}" if row[2] else "Sem alunos"
    print(f"{row[0]<25} {row[1]<10} {media}")

# Turmas vazias
cursor.execute('''
    SELECT t.nome_turma
    FROM turmas t
    LEFT JOIN alunos a ON t.id_turma = a.id_turma
    WHERE a.id_aluno IS NULL
''')
vazias = cursor.fetchall()
if vazias:
    print(f"\nTurmas sem alunos: {' '.join(v[0] for v in vazias)}")

conexao.close()
```

5.4. RIGHT JOIN — Todos os Registros da Tabela da Direita

O **RIGHT JOIN** é o espelho do LEFT JOIN: retorna **todos** os registros da tabela à **direita**, com NULL onde não há correspondência à esquerda. *Nota: o SQLite não suporta RIGHT JOIN nativamente — use LEFT JOIN com as tabelas invertidas para obter o mesmo resultado.*

```
-- RIGHT JOIN padrão (SQL Server, PostgreSQL, MySQL)
SELECT a.nome, t.nome_turma
FROM alunos a
RIGHT JOIN turmas t ON a.id_turma = t.id_turma;

-- Equivalente no SQLite (inverte as tabelas no LEFT JOIN)
```

```

SELECT a.nome, t.nome_turma
FROM turmas t
LEFT JOIN alunos a ON t.id_turma = a.id_turma;
-- Resultado idêntico: todas as turmas, mesmo sem alunos

```

5.5. Comparativo Visual dos JOINS

Tipo de JOIN	Retorna	Registros sem par
INNER JOIN	Somente linhas com correspondência nos dois lados	Excluídos
LEFT JOIN	Todas as linhas da tabela ESQUERDA + correspondências da tabela direita	Spreadsheets
RIGHT JOIN	Todas as linhas da tabela DIREITA + correspondências da tabela esquerda	Apêndices
FULL OUTER JOIN *	Todas as linhas de AMBAS as tabelas	NULL nos lados sem par

* *SQLite não suporta FULL OUTER JOIN. Simule com UNION de LEFT JOIN + RIGHT JOIN (ou dois LEFT JOINS invertidos).*

5.6. JOINS com Filtros, Ordenação e Agregação

JOINS podem ser combinados com qualquer cláusula já conhecida:

[illegible]

6. CRUD COMPLETO NO TERMINAL COM PYTHON

CRUD é o acrônimo das quatro operações essenciais em qualquer sistema que persiste dados: **C**reate (inserir), **R**ead (consultar), **U**ppdate (atualizar) e **D**elele (remover). Vamos agora construir um sistema de linha de comando completo usando `input()` para interagir com o usuário.

6.1. Estrutura de um CRUD

Operação	SQL	Função Python sugerida
Create	INSERT INTO	criar_registro()
Read	SELECT	listar_registros() / buscar_por_id()
Update	UPDATE	atualizar_registro()
Delete	DELETE	remover_registro()

6.2. Camada de Banco de Dados — Módulo db.py

Boa prática: separe o acesso ao banco em funções independentes da interface com o usuário. Isso facilita manutenção e reaproveitamento do código.

[illegible]

[illegible]

6.3. Menu Interativo via input()

```
# arquivo: main.py
import db

def exibir_menu():
    print("\n" + "="*45)
    print("      SISTEMA DE GERENCIAMENTO DE ALUNOS")
    print("="*45)
    print(" [1] Listar alunos")
    print(" [2] Cadastrar novo aluno")
    print(" [3] Atualizar aluno")
    print(" [4] Remover aluno")
    print(" [5] Listar cursos")
    print(" [6] Cadastrar curso")
    print(" [0] Sair")
    print("="*45)
    return input("Escolha uma opção: ").strip()

def listar_alunos():
    alunos = db.listar_alunos()
    if not alunos:
        print("Nenhum aluno cadastrado.")
        return
    print(f"\n{'ID':<5} {'Nome':<25} {'Email':<28} {'Nota':<6} {'Curso'}")
    print("-"*75)
    for a in alunos:
        nota = f"{a['nota']:.1f}" if a['nota'] else "N/A"
        curso = a['nome_curso'] or "Sem curso"
        email = a['email'] or "Sem e-mail"
        print(f"{'a['id_aluno']':<5} {a['nome']:<25} {email:<28} {nota:<6} {curso}")

def cadastrar_aluno():
    print("\n-- Cadastrar Aluno --")
    nome = input("Nome: ").strip()
    if not nome:
        print("Nome não pode ser vazio.")
        return
    email = input("E-mail (Enter para pular): ").strip() or None
    nota_str = input("Nota (Enter para pular): ").strip()
    nota = float(nota_str) if nota_str else None

    cursos = db.listar_cursos()
    if cursos:
        print("Cursos disponíveis:")
        for c in cursos:
            print(f"    [{c['id_curso']}] {c['nome_curso']}")
        id_str = input("ID do curso (Enter para nenhum): ").strip()
        id_curso = int(id_str) if id_str.isdigit() else None
    else:
        id_curso = None

    db.criar_aluno(nome, email, nota, id_curso)
    print(f"Aluno '{nome}' cadastrado com sucesso!")

def atualizar_aluno():
    listar_alunos()
    id_str = input("\nID do aluno a atualizar: ").strip()
    if not id_str.isdigit():
        print("ID inválido."); return

    aluno = db.buscar_aluno(int(id_str))
    if not aluno:
        print("Aluno não encontrado."); return

    print(f"Editando: {aluno['nome']}")
    nome = input(f"Novo nome [{aluno['nome']}] : ").strip() or aluno['nome']
    email = input(f"Novo e-mail [{aluno['email']}] : ").strip() or aluno['email']
    nota_atual = aluno['nota'] if aluno['nota'] else ""
    nota_str = input(f"Nova nota [{nota_atual}] : ").strip()
    nota = float(nota_str) if nota_str else aluno['nota']
    id_curso = aluno['id_curso']

    db.atualizar_aluno(int(id_str), nome, email, nota, id_curso)
    print("Aluno atualizado com sucesso!")

def remover_aluno():
    listar_alunos()
    id_str = input("\nID do aluno a remover: ").strip()
    if not id_str.isdigit():
```



```

        print("ID inválido."); return

aluno = db.buscar_aluno(int(id_str))
if not aluno:
    print("Aluno não encontrado."); return

conf = input(f"Confirma remoção de '{aluno['nome']}'? (s/n): ").strip().lower()
if conf == 's':
    db.remover_aluno(int(id_str))
    print("Aluno removido com sucesso.")
else:
    print("Operação cancelada.")

def cadastrar_curso():
    nome = input("\nNome do curso: ").strip()
    if nome:
        db.criar_curso(nome)
        print(f"Curso '{nome}' cadastrado!")
    else:
        print("Nome inválido.")

if __name__ == '__main__':
    db.criar_tabelas()
    while True:
        opcao = exibir_menu()
        if opcao == '1': listar_alunos()
        elif opcao == '2': cadastrar_aluno()
        elif opcao == '3': atualizar_aluno()
        elif opcao == '4': remover_aluno()
        elif opcao == '5':
            for c in db.listar_cursos():
                print(f" [{c['id_curso']}] {c['nome_curso']}")
        elif opcao == '6': cadastrar_curso()
        elif opcao == '0':
            print("Até logo!"); break
        else:
            print("Opção inválida.")

```

7. ATIVIDADE PRÁTICA: SISTEMA DE CONDOMÍNIO

Nesta atividade você construirá um sistema completo de gerenciamento de um condomínio com três tabelas relacionadas — **proprietários**, **apartamentos** e **garagens** — e um menu interativo no terminal com todas as operações de CRUD.

7.1. Modelagem das Tabelas

Veja o diagrama de relacionamentos abaixo e as restrições de cada tabela:

Tabela	Coluna	Tipo	Restrição	Descrição
proprietarios	id_proprietario	INTEGER	PK AUTOINCREMENT	Identificador único
proprietarios	nome	VARCHAR(100)	NOT NULL	Nome completo
proprietarios	cpf	VARCHAR(14)	UNIQUE	CPF formatado
proprietarios	telefone	VARCHAR(20)	—	Telefone de contato
proprietarios	email	VARCHAR(100)	UNIQUE	E-mail
apartamentos	id_apto	INTEGER	PK AUTOINCREMENT	Identificador único
apartamentos	numero	VARCHAR(10)	NOT NULL UNIQUE	Número/código do apto
apartamentos	andar	INTEGER	NOT NULL	Andar do edifício
apartamentos	quartos	INTEGER	DEFAULT 2	Qtd de quartos
apartamentos	area_m2	REAL	—	Área em m²
apartamentos	id_proprietario	INTEGER	FK → proprietarios	Dono atual
garagens	id_garagem	INTEGER	PK AUTOINCREMENT	Identificador único
garagens	numero_vaga	VARCHAR(10)	NOT NULL UNIQUE	Número da vaga
garagens	coberta	BOOLEAN	DEFAULT 1	1=coberta, 0=descoberta
garagens	id_apto	INTEGER	FK → apartamentos	Apto vinculado

7.2. Script Completo do Sistema de Condomínio

O código a seguir está dividido em três partes: (1) criação do banco e funções de dados, (2) funções de menu/interface e (3) loop principal.

Parte 1 — Banco de Dados e Funções CRUD

[illegible]

[illegible]

```
        return c.execute(
            'SELECT * FROM garagens WHERE id_garagem=?',(id_g,)).fetchone()

def atualizar_garagem(id_g, numero_vaga, coberta, id_apto):
    with conexaso() as c:
        c.execute(''
            UPDATE garagens
            SET numero_vaga=?, coberta=?, id_apto=?
            WHERE id_garagem=?
            '' , (numero_vaga, coberta, id_apto, id_g))

def remover_garagem(id_g):
    with conexaso() as c:
        c.execute('DELETE FROM garagens WHERE id_garagem=?',(id_g,))
```

Parte 2 — Funções de Interface (menus)

[illegible]

[illegible]

```

id_s = input("ID do apartamento (Enter para nenhum): ").strip()
id_a = int(id_s) if id_s.isdigit() else None
criar_garagem(vaga, coberta, id_a)
print("✓ Garagem cadastrada!")
elif op == '3':
    exibir_garagens()
    id_g = input("\nID a editar: ").strip()
    if not id_g.isdigit(): continue
    g = buscar_garagem(int(id_g))
    if not g: print("Não encontrado."); continue
    vaga = input(f"Vaga [{g['numero_vaga']}]: ").strip() or g['numero_vaga']
    cob = input(f"Coberta(1/0)[{g['coberta']}]: ").strip()
    coberta = int(cob) if cob in ('0', '1') else g['coberta']
    id_s = input(f"ID apto [{g['id_apto']}]: ").strip()
    id_a = int(id_s) if id_s.isdigit() else g['id_apto']
    atualizar_garagem(int(id_g), vaga, coberta, id_a)
    print("✓ Atualizado!")
elif op == '4':
    exibir_garagens()
    id_g = input("\nID a remover: ").strip()
    if not id_g.isdigit(): continue
    g = buscar_garagem(int(id_g))
    if not g: print("Não encontrado."); continue
    conf = input(f"Remover vaga '{g['numero_vaga']}'? (s/n): ").lower()
    if conf == 's':
        remover_garagem(int(id_g))
        print("✓ Removida!")

```


Parte 3 — Relatórios e Loop Principal

[illegible]

Como executar: Salve tudo em um único arquivo `condominio.py` (ou separe em `db.py` + `main.py`) e execute com `python condominio.py` no terminal.

Desafios Extras

Desafio 1: Adicionar validação de CPF (formato 000.000.000-00) antes de salvar.

Desafio 2: Criar função de busca por nome de proprietário usando LIKE.

Desafio 3: Adicionar tabela de *ocorrências* vinculada ao apartamento e criar CRUD para ela.

Desafio 4: Gerar relatório de apartamentos por andar mostrando taxa de ocupação.

Desafio 5: Exportar o relatório geral para um arquivo .txt.

Desafio 6: Implementar transações — se a criação do apartamento falhar, desfazer automaticamente a vinculação com o proprietário.

8. REFERÊNCIAS E PRÓXIMOS PASSOS

Documentação e Leitura Complementar

- SQLite Foreign Keys: <https://www.sqlite.org/foreignkeys.html>
- Python sqlite3 — Row Objects: <https://docs.python.org/3/library/sqlite3.html#sqlite3.Row>
- SQL JOINS — W3Schools: https://www.w3schools.com/sql/sql_join.asp
- Visual JOIN reference: <https://joins.spathon.com>
- Real Python — SQLite com Python: <https://realpython.com/python-sqlite-sqlalchemy/>

Próximos Tópicos

- **GROUP BY e HAVING:** agrupar resultados e filtrar grupos.
- **Subconsultas (Subqueries):** SELECT dentro de SELECT.
- **Views:** criar consultas reutilizáveis como se fossem tabelas.
- **Índices:** acelerar consultas em tabelas grandes.
- **Transações:** garantir consistência em operações múltiplas.
- **ORM com SQLAlchemy:** mapear tabelas para classes Python.
- **Migrações:** gerenciar evolução do esquema do banco.

Parabéns por chegar até aqui! Você agora domina o ciclo CRUD completo, relacionamentos entre tabelas e JOINS — o núcleo do desenvolvimento com bancos de dados relacionais. Continue praticando com projetos reais!